

Crypto Order Book Engine

A production-grade Rust system: design decisions, real-world analogies, and the math behind it.

This document is built alongside the code. Every decision has a reason. Every reason is grounded in how real markets actually work.

Part 0 - What Is an Order Book?

Imagine a farmers market where every vendor has a chalkboard showing what they will sell and at what price. Buyers walk around with their own boards showing what they want to buy. When a buyer's price matches a seller's price, a trade happens and both boards are erased. A stock or crypto exchange is exactly this - except it runs in RAM, processes millions of updates per second, and the boards change every millisecond.

The order book is the chalkboard. Our engine is the person watching that chalkboard continuously so that when you ask 'what is Bitcoin worth right now?', we answer in under a millisecond without calling Binance.



Figure 1 - Buyers, the exchange floor, and sellers. Our system mirrors this structure in code.

Why does this matter?

The gap between the best buy price and the best sell price is called the spread. It is the cost of doing business urgently. A tight spread means a liquid market - you can buy and sell quickly without moving the price. A wide spread means illiquidity. Every trading system in the world cares deeply about this number.

What we built is an engine that maintains a perfect real-time copy of Binance's order book in local memory. It connects over WebSocket, handles network failures automatically, validates every incoming message for correctness, and exposes a fast HTTP API so any service can query the current state of the market.

Part 1 - System Architecture

The system has four actors. Each runs independently in its own Tokio async task. They communicate only through typed channels and shared memory references. This is the actor model - a design pattern used by every major trading system.

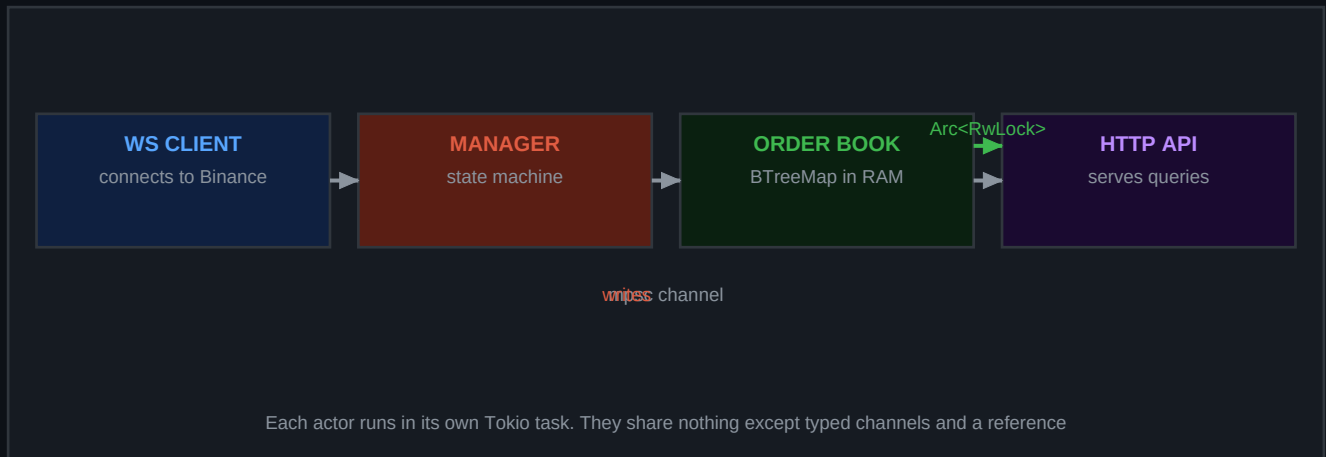


Figure 2 - The four actors. Arrows show data flow. Arc means shared read access.

Why separate actors?

If the WebSocket drops, only the WS Client task restarts. The order book keeps its data. The HTTP API keeps serving cached data. Each part can fail and recover without taking down the others.

The channel between the WS Client and Manager is a buffered queue with 10,000 slots. When the Manager goes offline to fetch a REST snapshot (which takes ~100ms), the WS Client keeps receiving messages and storing them in the buffer. Nothing is lost. This is the key insight of the whole design.

The Arc> shared between the Manager and the API is how we achieve concurrent reads with exclusive writes. Think of it as a shared whiteboard in a newsroom: anyone can read it at the same time, but only one person can write on it at a time, and they have to wait until all readers step back.

Part 2 - Why Rust?

Most financial systems are written in C++ or Java. C++ is fast but has no memory safety. Java is safe but has a garbage collector that can pause your program at any moment - unacceptable when a 50ms pause means you acted on stale data.

Rust gives you both: memory safety enforced at compile time, and zero pauses at runtime. The compiler rejects programs with data races before they ever run. This is not a hypothetical benefit - Rust is now used by Cloudflare, Discord, and several high-frequency trading firms precisely because of this.

Decision	We Chose	Alternative	Why We Chose It
Price type	rust_decimal	f64 float	f64 has silent rounding errors. $0.1 + 0.2 \neq 0.3$ in floating point. Financial systems cannot have invisible rounding in price calculations.
Async runtime	tokio	std threads	One OS thread per WebSocket connection does not scale. Tokio multiplexes thousands of async tasks onto a handful of threads with near-zero overhead.
HTTP framework	axum	actix-web	actix-web uses its own actor runtime that conflicts with Tokio. axum is built on top of Tokio natively - no impedance mismatch.
Error handling	thiserror	anyhow	anyhow erases error types into a single blob. We need to inspect each variant to decide: reconnect, skip, or resync. thiserror preserves that structure.
Logging	tracing	println!	println! has no concept of which async task is running. tracing attaches structured context (task ID, symbol, span) to every log line automatically.
WebSocket	tokio-tungstenite	raw TCP	WebSocket framing, masking, and ping/pong handling is hundreds of lines of protocol code. tokio-tungstenite does all of it, tested and correct.

Part 3 - Error Taxonomy

Most systems use one error type for everything and then write giant if-else chains to handle them. We use a typed enum where each variant encodes its own recovery strategy. The compiler forces you to handle every case.

This is not academic tidiness. It is engineering discipline. When a SequenceGap arrives at 3am and wakes someone up, the error message tells them exactly what happened and why.

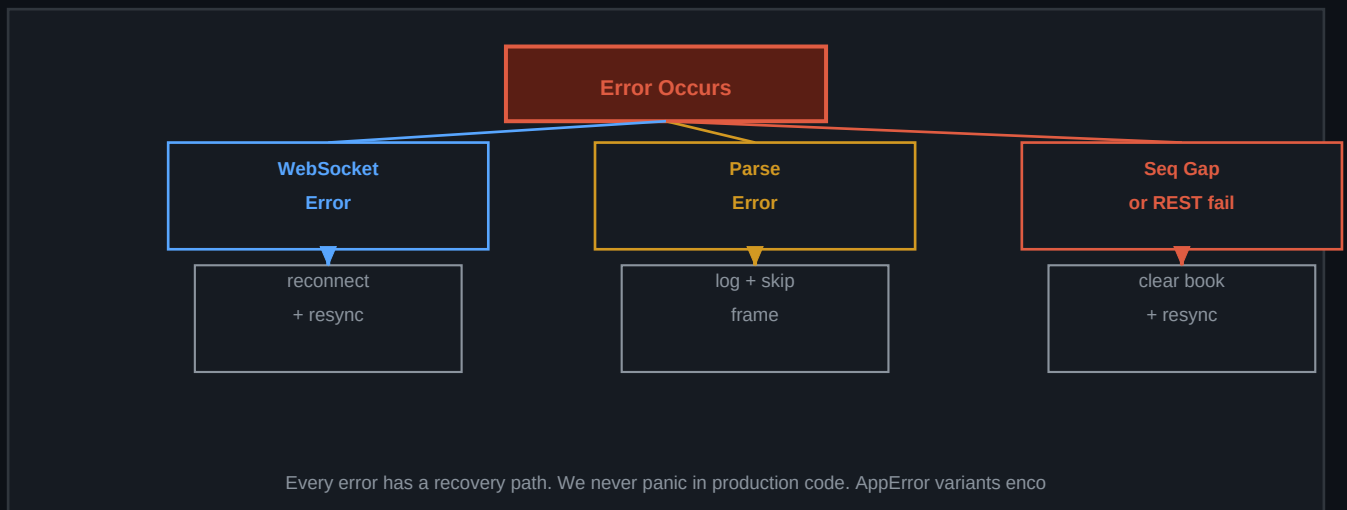


Figure 3 - Every error has a path. No error is fatal unless it reaches main().

The error variants:

WebSocket(tungstenite::Error) - The network dropped. Recovery: reconnect with backoff, send Reconnected signal, trigger resync.

Parse(serde_json::Error) - Binance sent a malformed JSON frame. Recovery: log it, skip that frame, keep going. One bad frame does not invalidate the whole stream.

InvalidField(String) - A price or quantity field is not a valid decimal. Same recovery as Parse.

SequenceGap { expected, got } - We received sequence 112 when we expected 111. The missed event cannot be recovered from the stream. Recovery: clear the book, restart the sync cycle from the snapshot.

SyncFailed - The first event we received after a snapshot does not connect cleanly to that snapshot. Recovery: same as SequenceGap - full resync.

Part 4 - The WebSocket Client

A WebSocket is like a phone call you make to Binance and then leave open permanently. Binance reads market updates into the phone continuously. Your job is to listen and never hang up. If the call drops, you call back immediately. If it keeps dropping, you wait a bit longer each time.

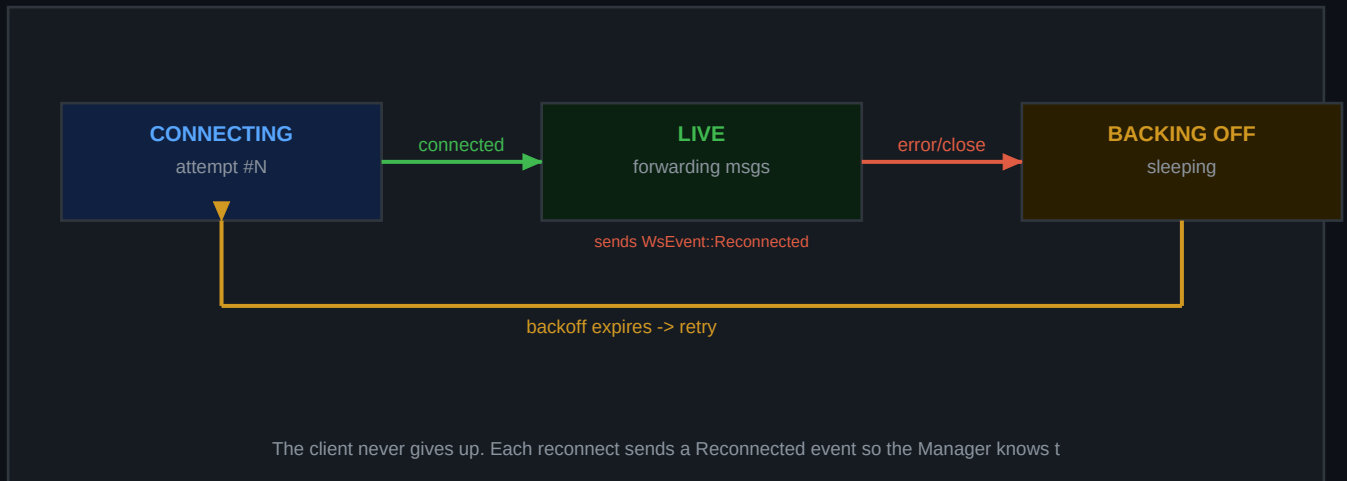


Figure 4 - The WS client state machine. It stays alive as long as the process runs.

Exponential backoff with jitter

When a connection fails, we wait before retrying. The wait doubles each time: 1s, 2s, 4s, 8s, up to 30s. This is exponential backoff. The reason: if Binance's servers are restarting, a thousand clients all reconnecting simultaneously would create a thundering herd - a flood that prevents the server from recovering.

Jitter adds a random amount (up to 1 second) to each wait. This spreads the reconnect attempts across time so no two clients retry at exactly the same moment.

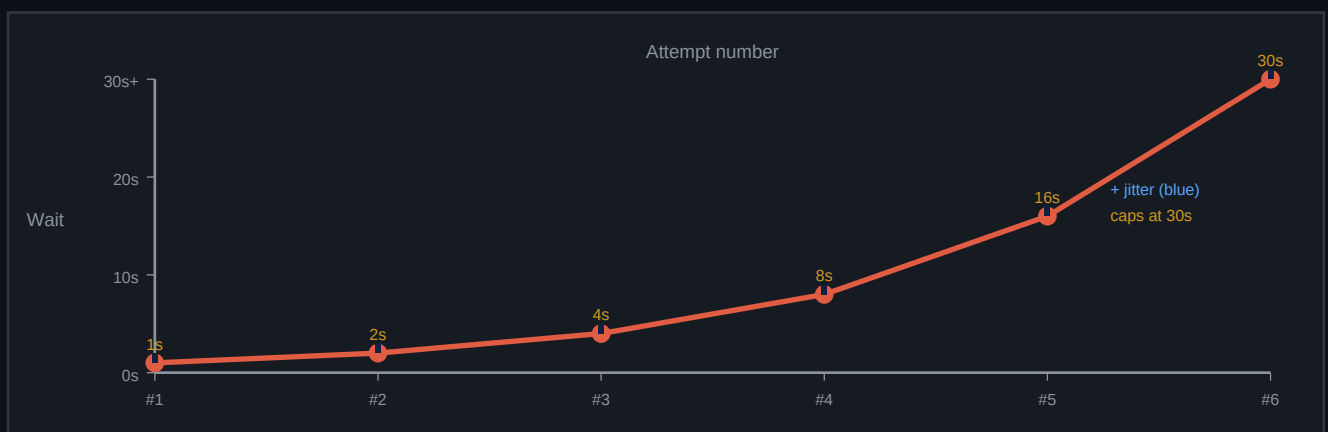


Figure 5 - Backoff curve. Capped at 30s. Blue bars show the jitter range per attempt.

The Reconnected signal

When we reconnect, the first thing we send into the channel is `WsEvent::Reconnected` - before any messages. The Manager sees this and knows: 'everything I buffered from the old connection is stale. Throw it away and start a fresh sync cycle.'

This is crucial because after a reconnect, the first messages in the buffer might have sequence numbers that pre-date the disconnect. Applying them would silently corrupt the book.

Part 5 - The Binance Sync Protocol

You cannot just connect to Binance's depth stream and start applying updates. The stream is a continuous diff feed - it tells you what changed, not what the current state is. If you join mid-stream, you have no baseline to apply the diffs to.

Think of it like joining a running commentary of a chess game. The commentator says 'white moved pawn from e2 to e4'. If you did not see the board before that move, you cannot reconstruct the current position.

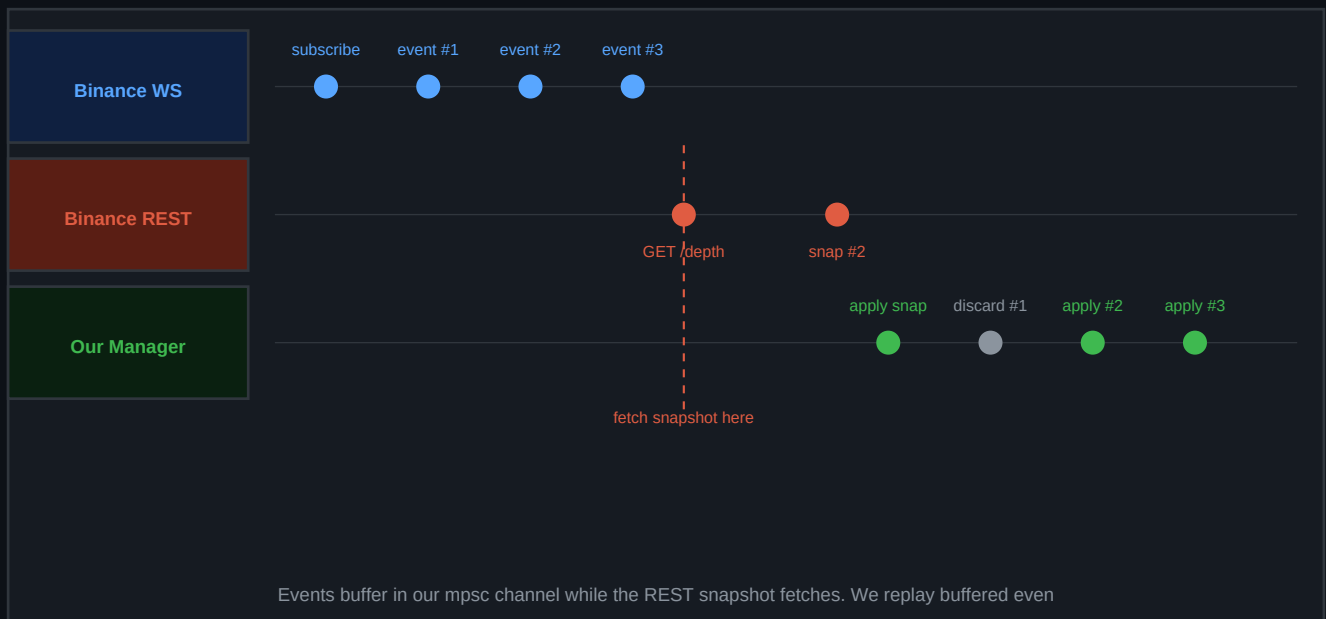


Figure 6 - The sync timeline. Events buffer in our channel while we fetch the REST snapshot.

The handshake algorithm:

Step 1: Open the WebSocket and start receiving events. They buffer in the mpsc channel.

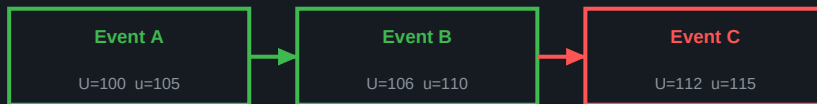
Step 2: Separately, call Binance's REST API to get a full snapshot. This snapshot has a lastUpdateId.

Step 3: Apply the snapshot to the book. This is our baseline.

Step 4: Drain the buffered events. Discard any event where `last_update_id <= snapshot_id` (already included in snapshot).

Step 5: The first event where `first_update_id <= snapshot_id + 1` AND `last_update_id > snapshot_id` is our connection point. Apply it.

Step 6: From here, each event's `first_update_id` must equal the previous event's `last_update_id + 1`. A gap means we lost data. Resync.



GAP! 111 missing
-> trigger resync

Like page numbers in a book. If page 111 is missing, you cannot understand page

Figure 7 - Sequence number continuity check. A gap triggers a full resync.

Scenario	Detection	Recovery
Normal live update	$U == \text{prev_u} + 1$	Apply to book, update prev_u
Stale event (in snapshot)	$u \leq \text{snapshot_id}$	Discard silently
Gap in handshake	$U > \text{snapshot_id} + 1$	Clear book, restart sync cycle
Gap in live mode	$U \neq \text{prev_u} + 1$	Clear book, restart sync cycle
WebSocket drops	WsEvent::Reconnected received	Clear book, restart sync cycle
REST snapshot fails	HTTP error or bad JSON	Retry with exponential backoff

Part 6 - Two-Layer Parsing

Binance's JSON format has field names like 'U', 'u', 'b', 'a'. Those are Binance's internal names. If we used those names throughout our codebase, every module would know about Binance's naming convention. If Binance ever changes their API, we would have to change code everywhere.

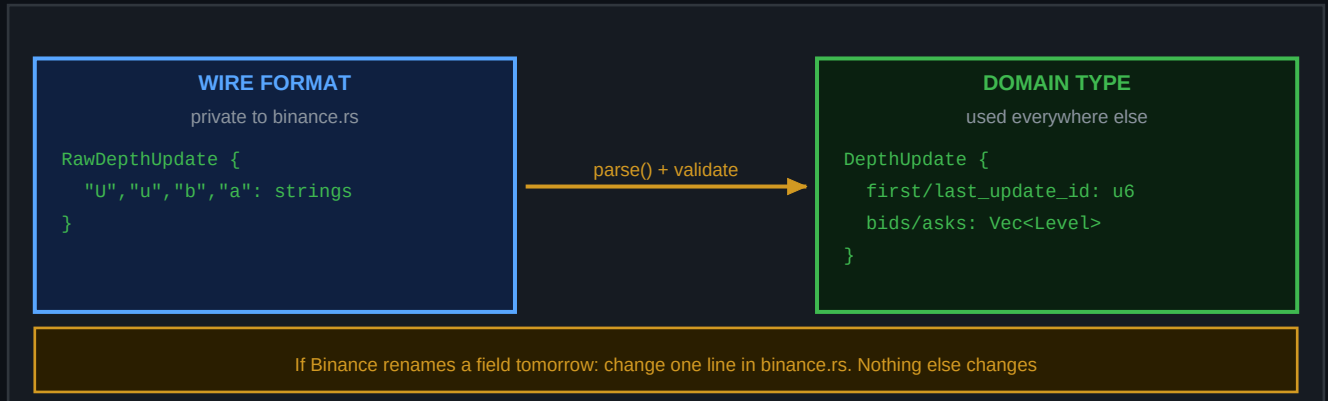


Figure 8 - Wire format is private to binance.rs. Domain type is what the rest of the system uses.

The two-layer pattern:

Layer 1 (private): `RawDepthUpdate` matches Binance's exact JSON. Field names are 'U', 'u', 'b', 'a'. This struct lives only in `binance.rs`.

Layer 2 (public): `DepthUpdate` uses readable names: `first_update_id`, `last_update_id`, `bids`, `asks`. All types are validated Decimal values, not raw strings.

The `parse()` function in between is the only place that knows Binance's format. It validates every field and returns an `AppError` if anything is malformed. The rest of the system works with clean domain types.

Part 7 - The Order Book Data Structure

We store bids and asks in a BTreeMap: price -> quantity. BTreeMap keeps its keys sorted at all times, always. Insert a new price, it lands in the right place. Remove a price, the others shift to fill the gap. This is not free - each operation is $O(\log n)$ - but for an order book it is exactly what we need.

BIDS (buyers)			ASKS (sellers)	
BEST BID			BEST ASK	
50,300	1.24	spread = 50	50,350	0.33
50,250	0.87		50,400	1.90
50,200	2.10		50,450	0.72
50,150	0.55		50,500	1.05

BTreeMap keeps prices sorted automatically. best_bid = last entry. best_ask = first entry

Figure 9 - BTreeMap layout for bids and asks. Best bid = last entry. Best ask = first entry.

Why not a HashMap?

A HashMap is $O(1)$ for lookup but unordered. To find the best bid with a HashMap, you would have to scan every entry. A BTreeMap's last() is $O(\log n)$ for the maximum key. For a book with 5000 price levels, $\log_2(5000)$ is about 12 operations. Fast enough.

The zero-quantity deletion signal:

Binance does not send a 'DELETE price level X' message. Instead, it sends a normal update for that price with a quantity of zero. Our is_removal() method checks for this. If we missed this signal, deleted price levels would stay in our book forever as ghost entries.

BEFORE			AFTER	
50300	1.24	qty=0 →	50300	1.24
50299	0.87		50299	--- DELETED ---
50298	2.10		50298	2.10
50297	0.55		50297	0.55

Binance does not send a 'delete' command. A quantity of zero IS the delete. Our is_removal

Figure 10 - Before and after a zero-quantity update. Price 50,299 is removed from the book.

Part 8 - Market Microstructure Signals

An order book is not just a list of prices. It is a map of market psychology. Every level, every quantity, every update tells you something about what buyers and sellers believe Bitcoin is worth and how urgently they want to trade.

Professional trading desks compute signals from the book continuously. We have implemented three of the most important ones.

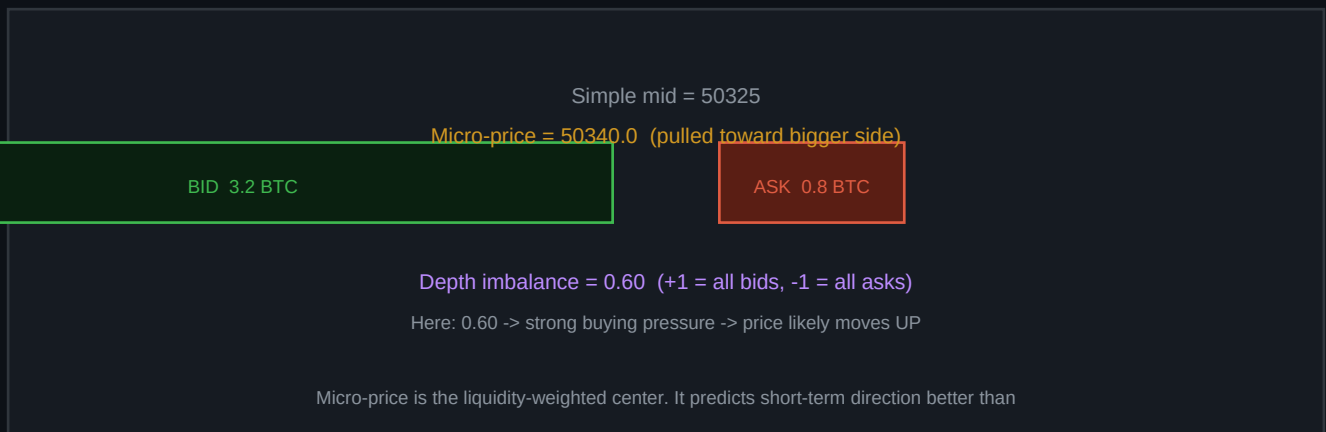


Figure 11 - Micro-price pulls toward the side with more liquidity. Depth imbalance quantifies the pressure.

Spread - the cost of urgency

The spread is the difference between the best ask and the best bid. If you want to buy Bitcoin right now, you pay the ask. If you want to sell right now, you get the bid. The spread is what you lose by acting immediately instead of waiting. Market makers earn the spread. Takers pay it.

Micro-price (weighted mid-price)

The arithmetic midpoint $(bid + ask) / 2$ treats both sides equally. But if there are 3 BTC waiting to buy and only 0.5 BTC waiting to sell, the market is more likely to move up (buyers outnumber sellers). Micro-price weights the midpoint by the opposite side's quantity. It predicts short-term price direction better than the simple midpoint.

The formula: $micro_price = (ask_price * bid_qty + bid_price * ask_qty) / (bid_qty + ask_qty)$

Depth Imbalance

Imbalance = $(bid_qty - ask_qty) / (bid_qty + ask_qty)$. Range: -1 (all sellers, price likely falls) to +1 (all buyers, price likely rises). Values above 0.3 or below -0.3 are considered strong signals by most market microstructure models.

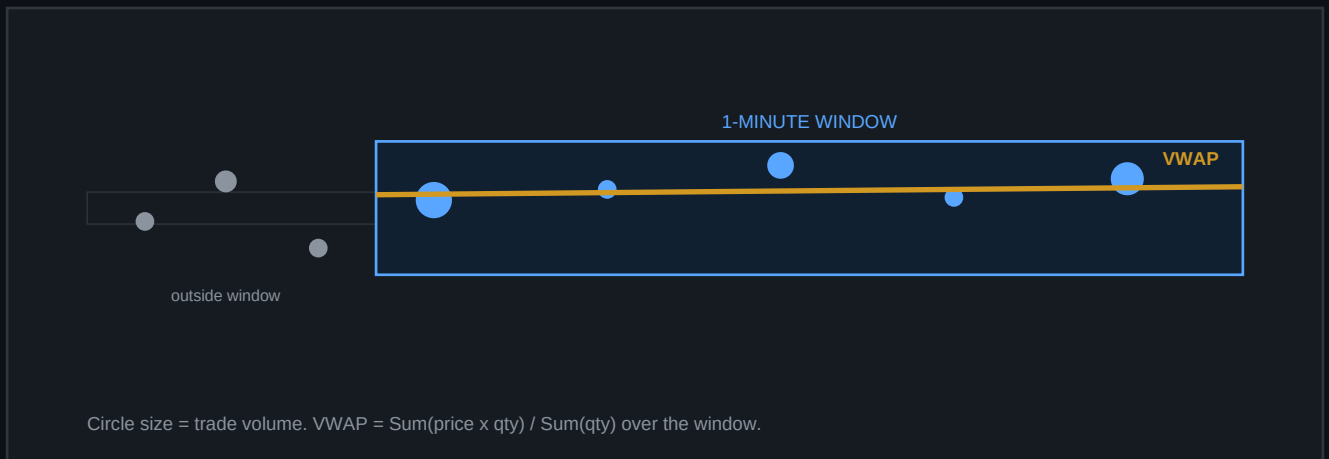


Figure 12 - Rolling VWAP window. Each circle is a trade. Size reflects volume. Grey = outside window.

VWAP - Volume Weighted Average Price

Every institutional fund benchmarks their execution against VWAP. If you bought Bitcoin at a price above the day's VWAP, you paid more than average. If you bought below, you were a good executor. We compute a 1-minute rolling VWAP from Binance's trade stream - a separate WebSocket subscription that delivers every individual trade in real time.

```
VWAP formula:  $\frac{\sum(\text{price}_i \times \text{quantity}_i)}{\sum(\text{quantity}_i)}$  over the window
```

We implement this with a VecDeque ring buffer that evicts old trades as the window slides forward. The sum of price*quantity and sum of quantity are maintained incrementally so each new trade is $O(1)$ to add and each evicted trade is $O(1)$ to remove.

Part 9 - The Manager State Machine

The Manager is the most complex part of the system. It is a state machine with three phases, and any error in any phase sends it back to the start. Think of a newspaper archivist who must always have the complete, up-to-date archive. If a batch of papers is lost, they do not try to reconstruct it. They get the current edition and start fresh.

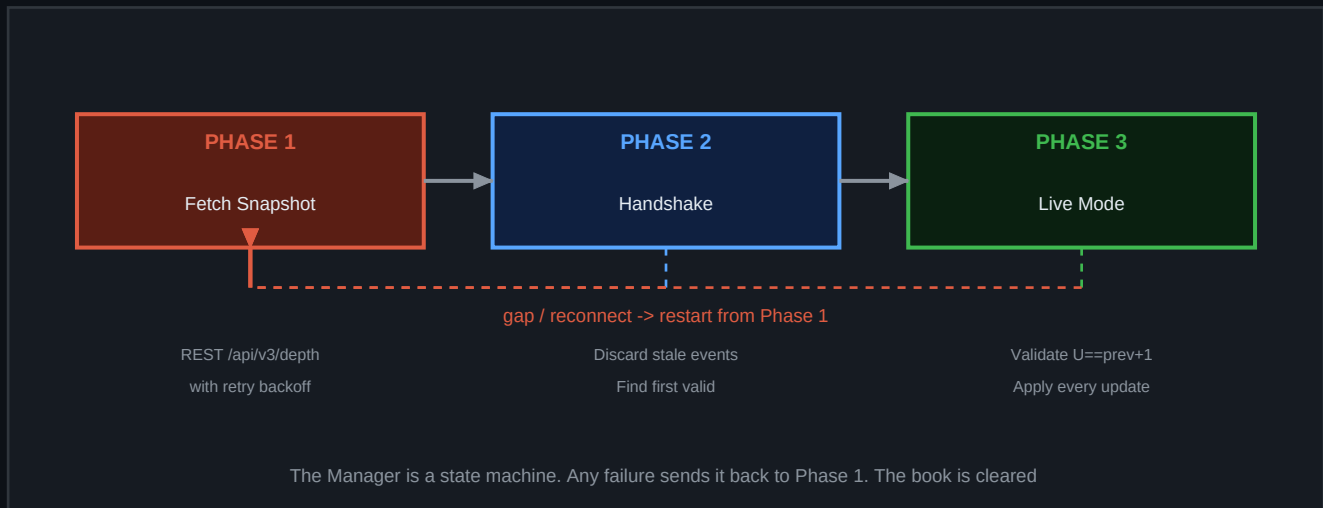


Figure 13 - The Manager state machine. Any error in Phase 2 or 3 restarts from Phase 1.

Phase 1: Snapshot Fetch

Call Binance's REST API: GET /api/v3/depth?symbol=BTCUSDT&limit;=5000. This returns up to 5000 price levels on each side and a lastUpdateId. If the request fails, we retry with exponential backoff. We never give up. The WebSocket client is still running and buffering events throughout.

Phase 2: Handshake

Drain the buffered events until we find the first one that connects to the snapshot's lastUpdateId. Apply that event. Return its lastUpdateId as the starting point for Phase 3. If we receive a Reconnected signal during this phase, the buffer is poisoned - restart.

Phase 3: Live Mode

Process events indefinitely. For each event, check that its first_update_id equals the previous event's last_update_id + 1. If yes, apply and advance. If no, we missed at least one event. Clear the book and go back to Phase 1.

Why std::sync::RwLock instead of tokio::sync::RwLock?

Because we never await inside the lock. The book operations (apply, read) are pure in-memory computation with no I/O. std::sync::RwLock is faster for synchronous critical sections because it does not need to interact with the async scheduler. This is one of the subtler Rust async decisions.

The Arc wraps the RwLock because both the Manager (writer) and the API handlers (readers) need to hold a reference to the same book across async task boundaries. Arc provides shared ownership with reference counting.

Part 10 - Metrics and Observability

A system you cannot observe is a system you cannot operate. In production, you will not be watching logs. You will be watching dashboards. When something goes wrong at 3am, you need to know whether it happened because of a network issue (reconnect counter spiking), a Binance problem (snapshot fetch errors), or a logic bug (sequence gap counter non-zero).

We expose all metrics in Prometheus format at GET /metrics. Prometheus scrapes it every 15 seconds. Grafana graphs it. PagerDuty alerts when thresholds breach.

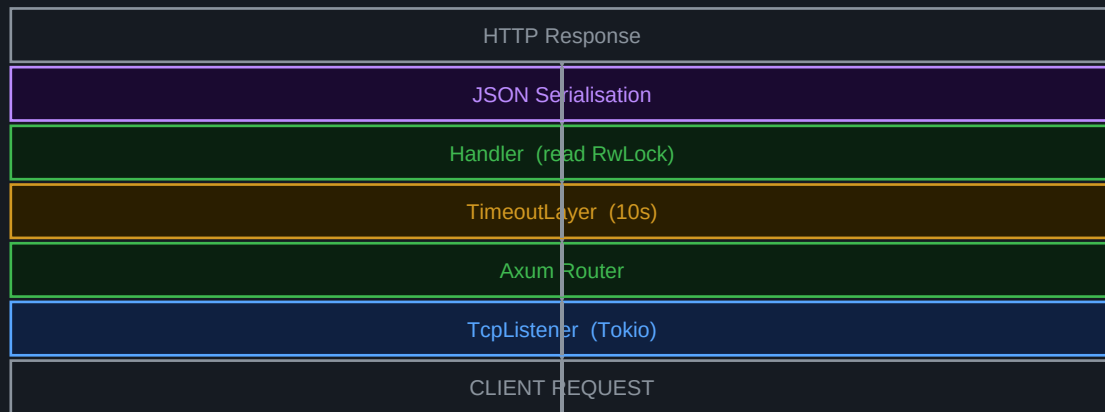


Figure 15 - The three Prometheus metric types and what we use each for.

Metric Name	Type	What a spike means
orderbook_ws_reconnects_total	Counter	Network instability or Binance outage. Should be near-zero in healthy operation.
orderbook_sequence_gaps_total	Counter	We are missing updates. Data integrity risk. Alert on any non-zero value.
orderbook_snapshots_total	Counter	Every gap triggers a new snapshot fetch. Correlated with sequence_gaps.
orderbook_updates_applied_total	Counter	Throughput. Should grow steadily. Plateau means the stream stopped.
orderbook_bid_depth	Gauge	If this drops to zero and stays there, the book is unsynced. Check health endpoint.
orderbook_spread	Gauge	Sudden spike = unusual market conditions. Worth alerting on if it persists.
orderbook_vwap_1m	Gauge	Current 1-minute VWAP. Compare to best_bid/best_ask to understand market direction.

Part 11 - The HTTP API

The API is intentionally simple: read the book, serialize to JSON, return. No business logic. No writes. The only state it touches is the read side of the RwLock.



Requests flow downward through each layer. The TimeoutLayer kills slow requests before the

Figure 16 - HTTP request flow through the middleware stack.

Endpoint	Returns	Use case
GET /health	200 if synced, 503 if syncing	Load balancer health check. Do not route traffic until this returns 200.
GET /book/best	best_bid, best_ask, spread, last_update_id	The single most queried endpoint. What is the price right now?
GET /book/snapshot	top N bids and asks (default 20, max 100)	Market depth display. Shows where volume is sitting.
GET /book/midprice	Micro-price (liquidity weighted)	Better than (bid+ask)/2 for estimating true fair value.
GET /book/imbalance	Imbalance ratio and interpretation string	Buy vs sell pressure signal. Range -1 to +1.
GET /book/vwap	1-minute rolling VWAP from trade stream	Execution benchmark. Compare your fill price against this.
GET /metrics	Prometheus text format	Scraped by Prometheus. Graphed in Grafana. Drives alerts.

The AppState pattern:

Axum clones AppState into every request handler. Because AppState only holds an Arc (reference-counted pointer), the clone is $O(1)$ - it just increments a counter. The actual order book is never copied. This is how we serve thousands of concurrent requests with a single in-memory book.

Timeout middleware:

Every request is wrapped in a 10-second timeout from tower-http's TimeoutLayer. If a handler stalls (for example, the RwLock is contended), the middleware kills the request and returns 408. This prevents a slow query from holding threads forever.

Part 12 - Configuration and Wiring

All configuration enters through environment variables, read once at startup. The Config struct is the single source of truth for every tunable parameter. This is the twelve-factor app principle: configuration in the environment.

Env Var	Default	What It Controls
SYMBOL	BTCUSDT	Which trading pair to track. Change to ETHUSDT to track Ethereum.
API_PORT	3000	Which port to bind the HTTP server on.
WS_BASE	wss://stream.binance.com:9443/ws/{symbol}@depth	WebSocket URL. Override for testnet or a different exchange.
TRADE_WS_BASE	wss://stream.binance.com:9443/ws/{symbol}@trade	Trade stream URL for VWAP calculation.
REST_BASE	https://api.binance.com	REST API base URL. Override for Binance testnet.
CHANNEL_BUFFER	10000	How many WS events to buffer during snapshot fetch. 10k = ~10 seconds of Binance updates.
RUST_LOG	info	Log verbosity. Set to debug for every event. info for normal ops.

How main.rs wires everything together:

main() is intentionally short. It reads config, builds shared state, spawns background tasks, and then drives the HTTP server in the foreground. When main() returns, the process exits. All background tasks are killed automatically because Tokio ties task lifetimes to the runtime.

1. Config::from_env() - read all configuration
2. Arc::new(RwLock::new(OrderBook::new())) - create the shared book
3. mpsc::channel(buffer) - create the event pipeline
4. ws::client::spawn(url, tx) - start depth WebSocket in background
5. ws::client::spawn(trade_url, trade_tx) - start trade WebSocket in background
6. tokio::spawn(Manager::new(rx, book, symbol).run()) - start the manager
7. tokio::spawn(TradeManager::new(trade_rx, vwap).run()) - start trade VWAP manager
8. metrics::init_all() - force-initialize all Prometheus counters to zero
9. axum::serve(listener, router(state)).await - serve HTTP until SIGTERM

Part 13 - Docker and Deployment

The Dockerfile has two stages. Stage 1 compiles the binary. Stage 2 runs it. Only the binary crosses the boundary. The final image contains no compiler, no Rust toolchain, no shell, no package manager. It is harder to exploit a system that has no tools to exploit with.

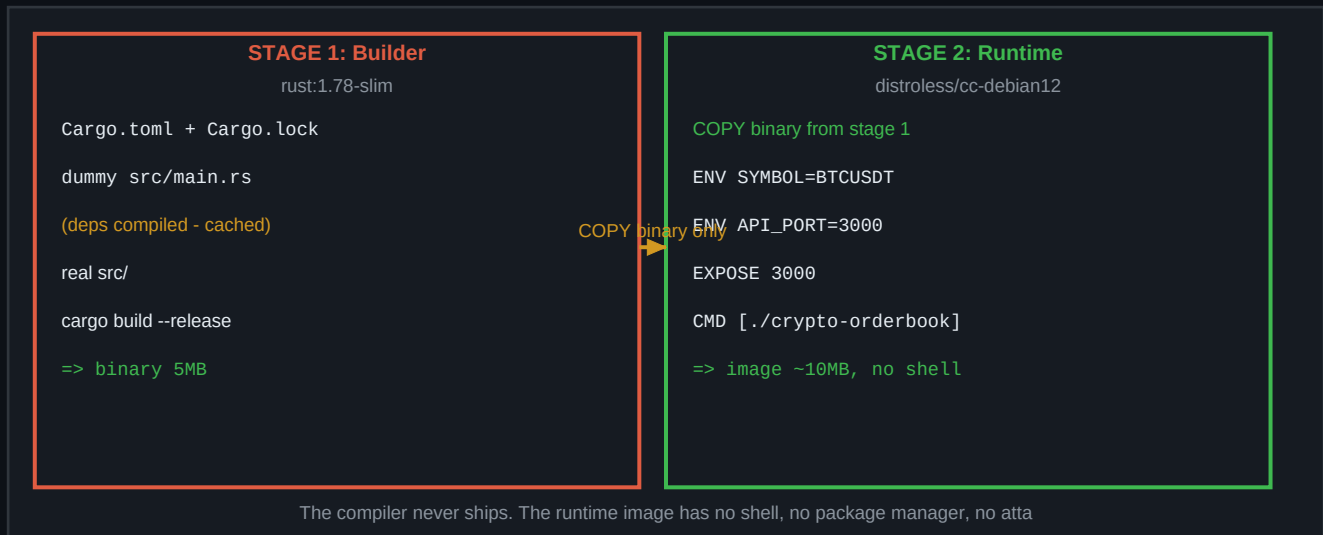


Figure 17 - Two-stage Docker build. Only the compiled binary ships to production.

Why distroless?

The gcr.io/distroless/cc-debian12 base image contains only the C standard library and its dependencies - the absolute minimum needed to run a compiled binary. No bash, no curl, no apt. If an attacker somehow breaks out of the binary, they land in an empty environment with no tools. This is the principle of least privilege.

Dependency layer caching:

We copy Cargo.toml and Cargo.lock before the source code and build a stub binary first. Docker caches each layer. If you change only src/ (not Cargo.toml), the dependency compilation layer is reused. This turns a 3-minute build into a 15-second build.

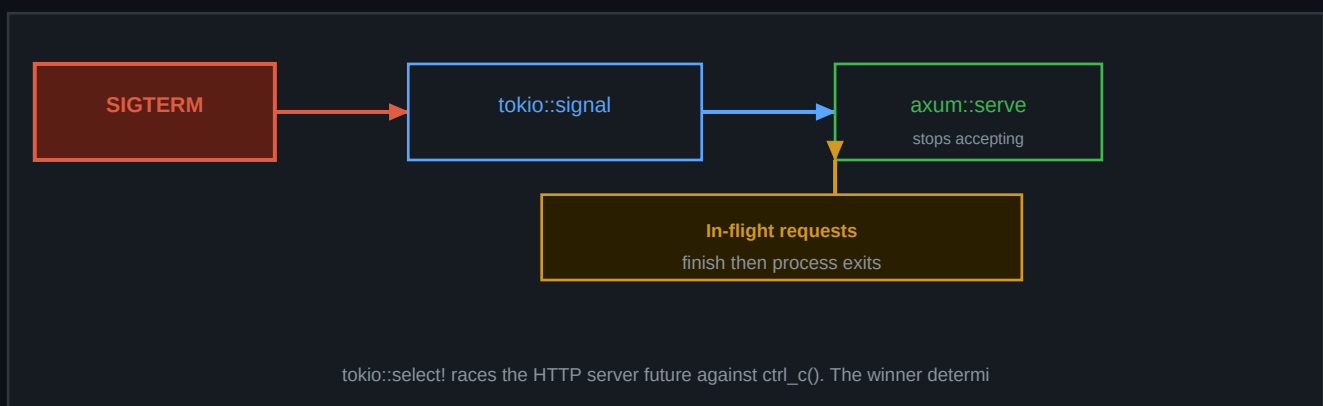


Figure 18 - Graceful shutdown. SIGTERM drains in-flight requests before the process exits.

Graceful shutdown prevents two problems:

1. Clients mid-request get a clean response instead of a broken connection.
2. If the Manager is mid-write to the book when SIGTERM arrives, the RwLock ensures the write completes before the process exits. We never write half an update.

Part 14 - Production Readiness Roadmap

What we have built is production-capable for a single symbol at moderate load. A real trading infrastructure system would add the following, roughly in priority order. Each item is a standalone engineering project.

Status	Feature	Why It Matters
BUILT	Prometheus /metrics	Counters + gauges live on every deployment
BUILT	VWAP Engine	1-minute rolling trade-weighted average pr
BUILT	Micro-Price	Liquidity-weighted midpoint - smarter than
BUILT	Depth Imbalance	Buy vs sell pressure signal used by HFT de
BUILT	Graceful Shutdown	SIGTERM drains in-flight before exit
BUILT	WebSocket Push API	broadcast::channel fan-out to subscribers
BUILT	Circuit Breaker	Stops hammering degraded Binance REST endp
BUILT	Integration Tests	8 tests hitting real Axum server on epheme
NEXT	Rate Limiter	Reject burst traffic before it hits the bo
PLANNED	TimescaleDB Writes	Persist every top-of-book snapshot for bac
PLANNED	Multi-Symbol	Fan out to N books, one manager per symbol

Figure 19 - Production roadmap. Green = built. Yellow = next. Red = planned.

Feature	Why It Matters in Production
WebSocket Push API	Polling REST adds latency and server load. A push model delivers updates to subscribers the moment the book changes. Every professional trading client uses WebSocket subscriptions.
Circuit Breaker for REST	If Binance's REST snapshot endpoint is degraded (slow but not failing), our retry backoff may not trigger. A circuit breaker opens after N consecutive slow responses and stops sending requests for a cooling period.
Rate Limiter on HTTP API	Without rate limiting, one runaway client can monopolize the RwLock read capacity. Tower's RateLimit layer allows N requests per second per IP before returning 429.
TimescaleDB Persistence	Every top-of-book snapshot, every spread measurement, every VWAP datapoint is a time-series record. TimescaleDB (PostgreSQL extension) stores billions of rows efficiently. This is the foundation for backtesting and strategy research.
Multi-Symbol Fan-out	One Manager per symbol, shared thread pool, separate books. The current design scales to this with minimal changes: Config becomes a Vec, and main spawns N manager pairs.
Risk Engine Hook	Every time the book updates, emit an event to a risk engine that checks open positions against current market price. If exposure exceeds limits, send a kill-switch signal. This is regulatory requirement in most jurisdictions.

Part 15 - WebSocket Push Server

Polling REST is fine for a dashboard that refreshes every second. But a trading algorithm needs to act on every single book update the moment it arrives. A WebSocket push server eliminates the poll cycle entirely. The book update travels from Binance -> our Manager -> every connected client in one unbroken async pipeline.

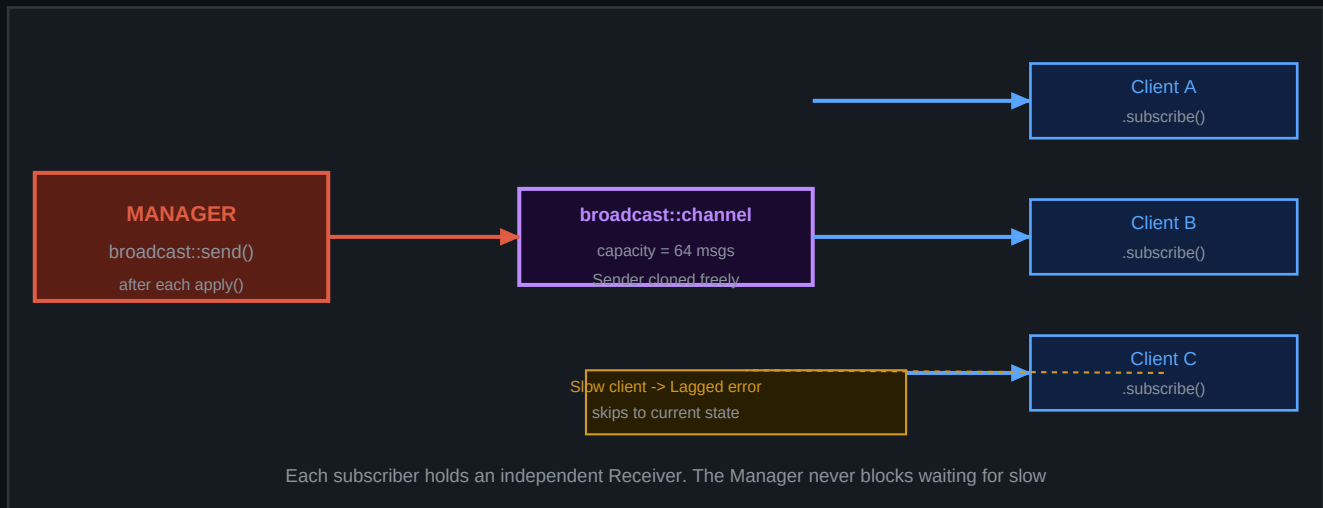


Figure 20 - Broadcast fan-out. One Manager publishes. Every subscriber receives independently.

Why broadcast::channel over mpvc?

mpvc is one sender, one receiver. If we used mpvc and wanted 10 clients, we would need 10 channels and the Manager would have to iterate through them. broadcast::channel is one sender, many receivers. Each subscriber calls .subscribe() to get their own independent Receiver. The channel internally manages fan-out. The Manager sends once. All 10 clients get it.

The Lagged error:

If a client is slow (perhaps it is on a weak connection or doing heavy processing), its receiver buffer fills up. When it is full, the oldest unread messages are dropped and the Receiver returns a Lagged(n) error indicating how many were skipped. For a market data display, this is acceptable - the next message will contain the current state anyway. We do not buffer indefinitely and we never slow down the Manager waiting for a slow client.

How to connect:

GET /ws/book - upgrade to WebSocket. Receive JSON BookPush events at the rate the book updates.

Part 16 - Circuit Breaker

The circuit breaker is named after the electrical safety device. When a circuit is drawing too much current, the breaker opens and stops the flow before it causes a fire. In software, the pattern stops outbound requests to a struggling service before your retries make the problem worse.

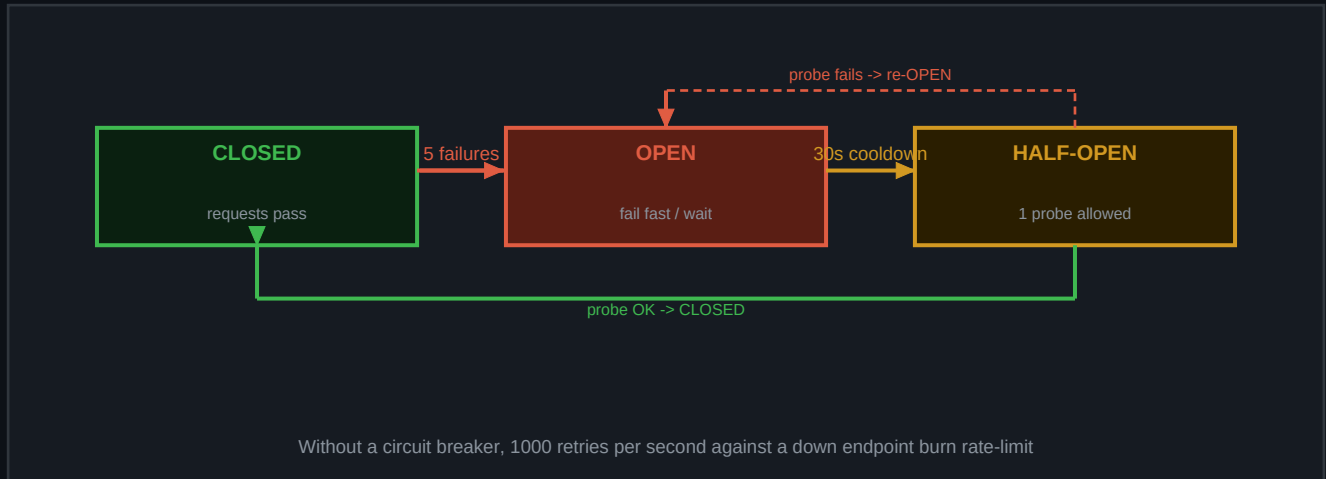


Figure 21 - Circuit breaker state machine for the Binance REST snapshot endpoint.

Why it matters for our system:

When Binance's REST API is degraded (slow but not fully down), our retry loop would fire every few seconds. Binance rate-limits by IP. If we burn through our rate-limit quota with retries, we get banned from the API - which means we cannot fetch snapshots even after the original problem is fixed. The circuit breaker stops outbound requests for 30 seconds after 5 consecutive failures, giving Binance time to recover and preserving our quota.

The three states:

Closed (normal): all requests pass through. Failure counter increments on each error, resets on success.

Open (failing): requests are rejected immediately without hitting Binance. After 30 seconds, transitions to HalfOpen.

HalfOpen (testing): exactly one probe request is allowed through. If it succeeds, circuit closes. If it fails, circuit re-opens.

Part 17 - Testing Strategy

The test suite has three layers. Each layer tests different things at different speeds. Together they give confidence that both individual functions and the full HTTP stack behave correctly.

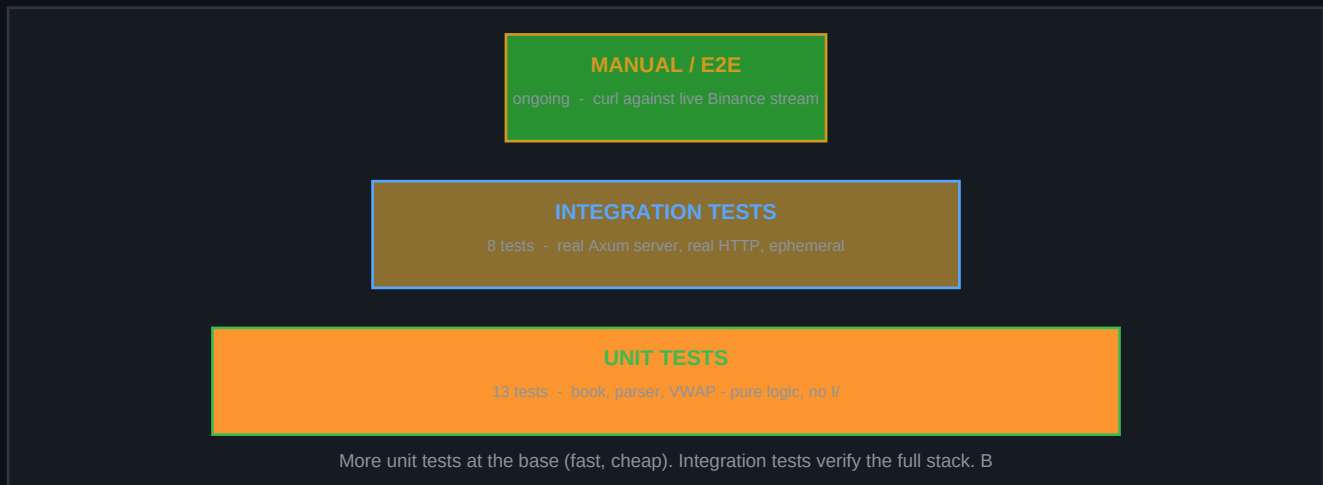


Figure 22 - Test pyramid. Fast unit tests at the base. Integration tests verify the full stack.

Unit tests (13 tests):

Pure logic tests with no I/O. Fast enough to run on every keystroke. They cover: BTreeMap ordering, zero-quantity deletion, spread calculation, micro-price and depth imbalance math, VWAP window sliding and eviction, Binance JSON parsing, and sequence ID validation.

Integration tests (8 tests):

Each test spins up a real Axum server on an ephemeral OS-assigned port (port 0 -> OS picks a free one). No mocks. Real HTTP requests via reqwest. They verify: health check returns 503 when unsynced and 200 when synced, best bid/ask/spread are correct, snapshot depth capping at 100, micro-price returns the arithmetic mid when quantities are equal, imbalance interprets heavy bids as 'strong buy pressure', and /metrics returns Prometheus text format with # HELP headers.

Why no mocked Binance WebSocket?

The WebSocket client and Manager run in background tasks that connect to external URLs. Testing them would require a mock WebSocket server. Instead, we test the book and parser logic in unit tests (which cover all the interesting cases), and test the HTTP API with a real server but a manually-populated book. This gives full coverage of both layers without the complexity of a mock exchange.

Part 18 - Latency Budget

Where does time actually go from 'Binance executes a trade' to 'our API responds'? Most systems guess. We can reason about it.

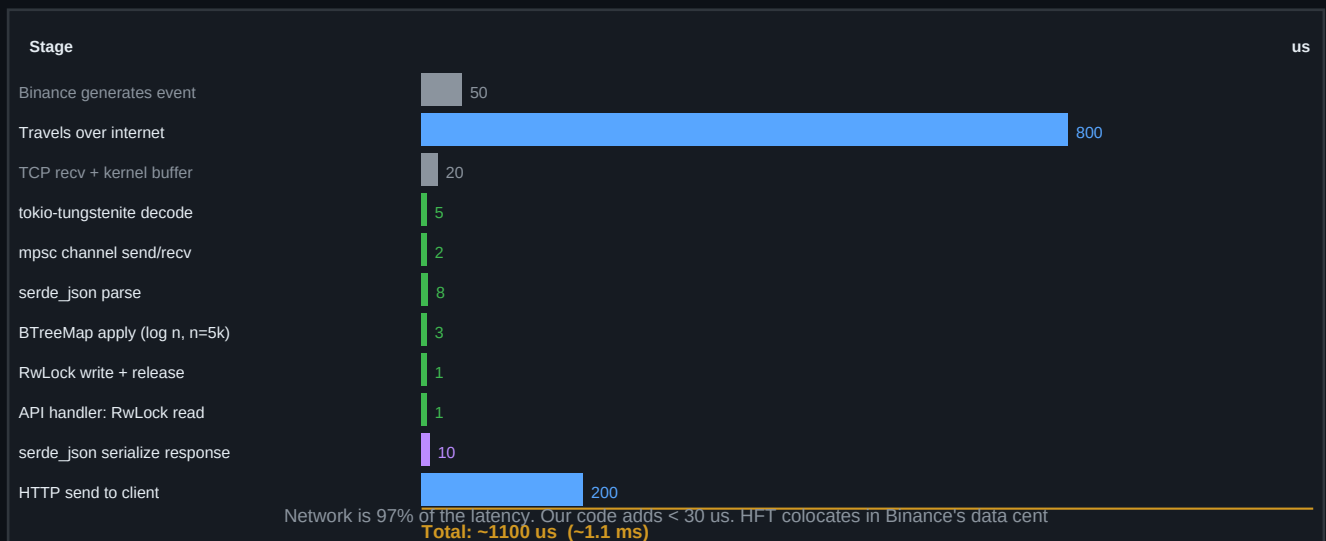


Figure 23 - Latency budget from Binance event to API response. Network dominates.

The key insight:

Network latency (the internet hop to Binance's servers) accounts for roughly 97% of total latency. Our code - parsing, applying updates, locking - adds under 30 microseconds. This is why high-frequency trading firms pay millions to colocate their servers in the same data center as the exchange. Cutting network latency from 800us to 20us is a 40x improvement in responsiveness. No amount of code optimization can match it.

What we can control:

tokio-tungstenite decode is already near-zero. serde_json is the hottest path in our code - if we needed to optimize further, we would switch to simd-json which is 3-5x faster. The BTreeMap apply is $O(\log n)$ where n is the number of price levels (~5000 for BTC). $\log_2(5000) = 12$ iterations. Negligible.

The RwLock:

std::sync::RwLock is contention-free when the write lock is not held. Since the Manager holds the write lock for less than 10 microseconds per update (pure memory operations), API handlers almost never wait. On Tokio's default 4-core thread pool, read requests can execute truly simultaneously on all 4 threads while the Manager writes.

Part 19 - Interview Preparation: Questions and Answers

These are the questions you will be asked. Every answer is grounded in this codebase.

Question	The Answer (from this code)
Why BTreeMap and not HashMap for the order book?	HashMap is $O(1)$ but unordered. To find best bid with HashMap, you scan every entry - $O(n)$. BTreeMap's <code>last()</code> gives the maximum key in $O(\log n)$. For 5000 price levels, $\log_2(5000) = 12$ operations. We also need in-order iteration for <code>top_bids()</code> which is $O(k)$ with BTreeMap and $O(n \log n)$ with HashMap. The sorted property is fundamental.
Why <code>std::sync::RwLock</code> instead of <code>tokio::sync::RwLock</code> ?	Because we never <code>.await</code> inside the lock. Book operations are pure in-memory computation. <code>tokio</code> 's <code>RwLock</code> is designed for async critical sections where you yield to the scheduler while holding the lock. Using it for synchronous work adds scheduling overhead for no benefit. <code>std::sync::RwLock</code> is faster here.
Why <code>Arc</code> for sharing the book?	<code>Arc</code> gives shared ownership across task boundaries - the Manager task and multiple API handler tasks all need a live reference to the same book. <code>RwLock</code> enforces the invariant: one writer, many concurrent readers. Without <code>Arc</code> , you cannot share a reference across <code>tokio::spawn</code> boundaries. Without <code>RwLock</code> , concurrent readers would race.
How does the Binance sync protocol work?	Subscribe to the WebSocket diff stream first. Events buffer in our mpsc channel. Separately, fetch a REST snapshot. Apply the snapshot. Scan buffered events and discard any where <code>last_update_id <= snapshot_id</code> (already included). Find the first event where <code>first_update_id <= snapshot_id+1</code> and apply it. From then, every event's <code>first_update_id</code> must equal the previous <code>last_update_id+1</code> . A gap triggers a full resync.
What happens when the WebSocket disconnects?	The client task's <code>drive_stream()</code> loop exits. It sends <code>WsEvent::Reconnected</code> before any new messages. The Manager receives <code>Reconnected</code> , clears the book, and restarts the sync cycle from Phase 1. In the meantime, the HTTP API returns stale data - <code>/health</code> returns 503 because <code>last_update_id</code> resets to 0 on <code>clear()</code> . The WS client backs off exponentially before reconnecting.
How do you handle a sequence gap?	In <code>run_live()</code> , we check that each event's <code>first_update_id == prev_last_id + 1</code> . If not, we return <code>LiveResult::SequenceGap</code> . The outer loop increments the <code>sequence_gaps</code> metric, clears the book, and restarts the sync cycle. We never try to reconstruct missing events because there is no API to fetch them individually.
Why mpsc channel with a 10,000-element buffer?	The snapshot REST fetch takes ~100ms. At peak, Binance sends ~100 depth updates per second. So we need to buffer ~10 events per snapshot fetch. 10,000 is 1000x headroom - it absorbs slow snapshot fetches, high-traffic bursts, and brief processing delays without dropping a single event. The buffer is the key to the sync protocol working correctly.

Question	The Answer (from this code)
What is micro-price and why is it better than $(bid+ask)/2$?	Micro-price weights each side by the other side's quantity: $(ask * bid_qty + bid * ask_qty) / (bid_qty + ask_qty)$. If there is 10x more buying pressure than selling pressure, the micro-price is 91% of the way toward the ask. This reflects where the market is likely to trade next. The arithmetic midpoint ignores this liquidity signal entirely.
How does the circuit breaker work?	Three states: Closed (requests pass), Open (fast-fail for 30s), HalfOpen (one probe). After 5 consecutive REST failures, the breaker opens. This prevents burning Binance's rate-limit quota with retries against a down endpoint, which would prevent recovery even after Binance recovers. After 30s cooldown, we let one probe through. Success closes the circuit. Failure re-opens it.
How does the broadcast::channel for WebSocket push work?	broadcast::channel has one Sender and arbitrarily many Receivers. Each WebSocket client calls .subscribe() to get their own Receiver. The Manager calls push_tx.send() once per book update. All clients receive the same payload independently. If a client is too slow, it receives a Lagged(n) error and skips to the current state - we never block the Manager waiting for slow clients.

What makes this system interview-ready:

Every choice in this codebase has a reason you can explain clearly. BTreeMap because we need sorted extremes. mpsc channel with large buffer because snapshot fetches take 100ms and we cannot drop 100ms worth of updates. std::sync::RwLock not tokio's because no await inside the lock. Two-layer parsing to isolate the Binance dependency. Exponential backoff with jitter to prevent thundering herds.

These are the answers senior engineers give. Not 'I used X because it was popular' but 'I used X because of specific property Y which this problem requires, and I considered alternative Z but rejected it because of tradeoff W.'

Built with Rust 1.78 + Tokio 1 + Axum 0.7 + Prometheus 0.13

github.com/Mattbusel/crypto-orderbook